

## REVISÃO PARA AVALIAÇÃO

### 1. Criando o projeto com Vite

Abra o terminal no VS Code e crie um novo projeto React com TypeScript usando Vite. Em seguida, rode o servidor local. Verifique se ao alterar o título no App.tsx, a atualização acontece instantaneamente no navegador sem recarregar a página inteira.

### 2. Criando a Store

No projeto criado, instale o Redux Toolkit e React-Redux. Crie um arquivo store.ts com a configuração inicial da Store utilizando configureStore. Importe a Store no main.tsx e use o Provider para englobar o componente <App />.

### 3. Criando um Slice de Usuários

- Crie um arquivo userSlice.ts e utilize createSlice para:
- Definir o estado inicial { users: [] }.
- Criar reducers addUser e removeUser.
- Exporte as actions e o reducer.
- Teste adicionando usuários manualmente no console usando store.dispatch(addUser(...)).

### 4. Listando Usuários no Componente

No componente App.tsx, utilize o hook useSelector para listar os usuários armazenados no estado global.

Mostre os nomes em uma <ul>.

Adicione um botão para disparar addUser com dados fictícios.

### 5. Criando um Slice de Hábitos

Crie um habitSlice.ts com os seguintes reducers:

- addHabit → adiciona um hábito (com id, nome e categoria).
- removeHabit → remove hábito pelo id.
- filterHabits → filtra hábitos por categoria.

Teste no console adicionando e filtrando hábitos.

### 6. Formulário de Cadastro de Hábitos

No App.tsx, crie um formulário com inputs para nome e categoria do hábito.

Ao enviar, dispare addHabit.

Liste os hábitos cadastrados abaixo do formulário.

## 7. Implementando Filtro de Hábitos

Adicione um `<select>` no `App.tsx` com opções de categorias (ex.: “Saúde”, “Estudo”, “Lazer”).

Ao escolher uma categoria, dispare `filterHabits`.

Exiba apenas os hábitos filtrados na lista.

## 8. Criando Hook Personalizado

Crie um hook `useUsers` que encapsule o uso de `useSelector` e `useDispatch` para manipular usuários.

Esse hook deve expor funções `getUsers`, `addUser` e `removeUser`.

Refatore o `App.tsx` para usar apenas o hook, deixando o código mais limpo.

## 9. Montando um Mini Dashboard

Monte uma tela simples com duas seções:

1. Gerenciar Usuários → cadastrar e listar usuários.
2. Gerenciar Hábitos → cadastrar, listar e filtrar hábitos.

Organize cada seção em um componente próprio.

Use apenas Redux Toolkit para o gerenciamento global.

## 10. Na integração com a API Alpha Vantage, por que os dados brutos recebidos precisam ser mapeados (`formattedData.map`) antes de serem exibidos?

- A) Porque a API envia dados criptografados em binário.
- B) Para transformar o objeto complexo da API em um array simples compatível com o `FlatList`.
- C) Para traduzir os nomes das ações do inglês para o português.
- D) Para deletar a chave da API automaticamente após o uso.
- E) Para aumentar a resolução das imagens recebidas.

## 11. Analise a lógica de classificação de IMC do material. Escreva uma estrutura `if/else if` que receba um valor `imc` e retorne a cor "red" para Obesidade Grau II (IMC entre 35.0 e 39.9) e a classificação correspondente.

## 12. Uma aplicação de Bolsa de Valores exibe um `ActivityIndicator` enquanto os dados estão sendo carregados. Explique como a variável de estado `loading` deve ser manipulada dentro da função `fetchStockData` (início, sucesso e erro) para que esse indicador funcione corretamente.

## 13. O que acontece se o desenvolvedor esquecer de passar o array de dependências vazio `[]` no `useEffect` que chama a API?

- A) A API nunca será chamada.

- B) A API será chamada apenas uma vez ao iniciar.
- C) A API entrará em um loop infinito de chamadas a cada atualização de estado.
- D) O aplicativo fechará com um erro de "Null Pointer".
- E) O componente se tornará invisível.

**14. No arquivo tsconfig.json, qual a importância da opção "target": "ES2015" e o que pode ocorrer se ela for definida como "ES5" em um projeto moderno?**

**15. No código de consumo de API, o que faz a função .slice(0, 10) aplicada aos dados recebidos?**

- A) Divide o valor das ações por 10.
- B) Limita a exibição apenas aos 10 primeiros registros da lista.
- C) Remove os 10 últimos caracteres de cada string.
- D) Multiplica o tempo de resposta por 10 milissegundos.
- E) Cria 10 cópias de segurança dos dados.

**16. Para que serve a propriedade keyExtractor em um FlatList?**

- A) Para definir a cor da fonte da lista.
- B) Para fornecer uma chave única para cada item, auxiliando o React na performance de renderização.
- C) Para extrair senhas de usuários logados.
- D) Para abrir o teclado numérico automaticamente.
- E) Para conectar a lista ao banco de dados Firebase.

**17. Explique como o componente <Button /> do React Native lida com eventos de clique e qual propriedade é usada para passar a função de execução (ex: calcular).**

**18. Sua aplicação precisa exibir a mensagem "Por gentileza, insira números válidos" caso o usuário deixe um campo de nota vazio. Como você implementaria essa validação e qual componente usaria para exibir esse erro em vermelho?**

**19. Qual a principal vantagem de usar o Axios em vez do Fetch API nativo, segundo o contexto de desenvolvimento apresentado?**

- A) O Axios é feito pela Microsoft e o Fetch pelo Google.
- B) O Axios já converte automaticamente a resposta para JSON e possui uma sintaxe mais limpa para requisições complexas.
- C) O Fetch não funciona em dispositivos Android.

D) O Axios permite criar interfaces gráficas 3D.

E) O Fetch é pago, enquanto o Axios é gratuito para estudantes

## 20. Manipulação de Estado e Tipagem

Analise o trecho de código abaixo, que faz parte de um componente de formulário:

```
const [valor, setValor] = useState("");

const verificarEntrada = (texto: string) => {
  setValor(texto);
};

// No retorno do componente:
<TextInput
  style={styles.input}
  onChangeText={verificarEntrada}
  value={valor}
  keyboardType="numeric"
/>
```

Explique a importância da propriedade `value={valor}` no componente `TextInput`. O que aconteceria com a interface do usuário se essa propriedade fosse removida, mesmo que a função `onChangeText` continuasse atualizando o estado?

## 21: Renderização Condicional e Lógica

Considere um aplicativo de cálculo de média escolar. O desenvolvedor deseja exibir a situação do aluno apenas após o botão "Calcular" ser pressionado.

Escreva um pequeno trecho de código em JSX que utilize o operador lógico `&&` ou um operador ternário para exibir um componente `<Text>` com a mensagem "Aprovado" apenas se a variável de estado `media` for maior ou igual a 6 e a variável `calculado` (booleana) for verdadeira. Explique por que essa técnica é preferível ao uso de múltiplos componentes de tela diferentes.

## 22. Hooks e Ciclo de Vida (useEffect)

No desenvolvimento de um app que consulta o valor de ações (como o exemplo da Alpha Vantage no PDF), é comum utilizar o seguinte padrão:

```
useEffect(() => {  
    fetchStockData();  
}, []);
```

O que representa o array vazio [] como segundo argumento do useEffect? Se um desenvolvedor colocasse uma variável de estado dentro desse array (ex: [ticker]), como o comportamento da aplicação mudaria em relação às requisições à API?

### 23. Estilização e Flexbox no React Native

No React Native, o layout é construído usando Flexbox. Observe o código de estilo abaixo:

```
const styles = StyleSheet.create({  
    container: {  
        flex: 1,  
        justifyContent: 'center',  
        alignItems: 'center',  
        backgroundColor: '#fff',  
    },  
});
```

Descreva o efeito visual que esse estilo causa em uma <View> que envolve todo o conteúdo da tela. Qual a diferença fundamental entre as propriedades justifyContent e alignItems quando o flexDirection padrão do React Native está ativo?

### 24. Integração com APIs e Tratamento de Erros

Ao utilizar a biblioteca axios para buscar dados de um arquivo JSON ou API externa, o desenvolvedor geralmente utiliza a estrutura try...catch.

```
const loadData = async () => {  
    try {  
        const response = await axios.get('https://api.exemplo.com/dados');  
        setDados(response.data);  
    } catch (error) {  
        console.error("Erro ao carregar:", error);  
        setMensagemErro("Não foi possível conectar ao servidor.");  
    }  
};
```

Explique por que é considerada uma boa prática de UX (Experiência do Usuário) utilizar o bloco `catch` para atualizar um estado de erro (como `setMensagemErro`) em vez de apenas exibir o erro no `console.error`. Como isso impacta a interação do usuário com o aplicativo móvel?

**25. Qual a finalidade do atributo `value` em um componente `<TextInput />`?**

- A) Definir o preço do aplicativo.
- B) Vincular o conteúdo visual do campo a uma variável de estado.
- C) Nomear o campo para o banco de dados.
- D) Criar uma borda colorida no campo.
- E) Impedir que o usuário digite caracteres especiais.

**26. Ao utilizar `axios.get(url)`, o que é necessário fazer para inserir variáveis dinâmicas (como um símbolo de ação) dentro da string da URL?**

- A) Usar aspas simples e o sinal de `'+'`.
- B) Usar Template Literals (crases) e a sintaxe `${variavel}`.
- C) Usar parênteses e vírgulas.
- D) Usar o comando `import variable`.
- E) Não é possível usar variáveis dinâmicas em URLs.

**27. No projeto da Calculadora de Médias, por que é importante verificar `if (isNaN(n1) ...)` antes de realizar o cálculo final?**

**28. O componente `<FlatList />` é preferível ao `<ScrollView />` quando:**

- A) Temos poucos elementos estáticos na tela.
- B) Precisamos exibir uma lista longa de dados de forma performática.
- C) Queremos mudar a cor de fundo da aplicação.
- D) Não queremos usar JavaScript no código.
- E) O aplicativo só funcionará em modo offline.

**29. Analise o seguinte trecho de código utilizado para buscar dados de uma API:**

```
const [dados, setDados] = useState([]);
const carregarDados = async () => {
  try {
    const res = await
    axios.get(' [https://api.exemplo.com/itens] (https://api.exemplo.com/itens) ');
    setDados(res.data.slice(0, 5));
```

```
    } catch (error) {  
      console.log(error);  
    }  
  };  
};
```

Qual será o conteúdo da variável de estado 'dados' após a execução bem-sucedida da função 'carregarDados'?

- A) Um objeto contendo o status da requisição e o erro, se houver.
- B) Um array contendo todos os itens retornados pela API.
- C) Uma string formatada com os 5 primeiros caracteres da resposta.
- D) Um array contendo apenas os 5 primeiros itens da resposta da API.

**30. No desenvolvimento com React Native, é comum a necessidade de exibir uma lista de dados provenientes de um arquivo JSON ou API. Ao utilizar o componente `<FlatList />`, qual propriedade é obrigatória para passar o array de objetos que será renderizado na tela?**

- A) `contentContainerStyle`
- B) `renderItem`
- C) `data`
- D) `keyExtractor`
- E) `ItemSeparatorComponent`